Theoretical minimum: Paper 006

AuthorityEffect: None. Read the fictional preface and study guide. Keep the instructor key closed until completing the questions. Give exact fractions, explicit source or encoding assumptions, and the strongest conclusion the evidence permits. These exercises concern the manuscript's audited commit, not every later branch.

Level 0: name the object before making a claim

Q01. Event, reason, and truth

A stored entry says `NO_NATIVE_PORT` and contains `{"phi1": "0"}`. Which parts of this record were computed by the sink? Can the record establish that the proposed mathematical witness is invalid, or that a native functional was zero? Identify the missing bindings.

Q02. Four persistence claims

An API exposes append, writes a temporary copy of all records, and calls `os.replace`. Distinguish logical append, physical immutability, successful process restart, and power-loss durability. Which are supported by the source and the focused restart tests?

Q03. A fixed marker

An entry and a tail receipt contain `authority_effect: "None"`. Is that value JSON `null`, a cryptographic signature, or an authorization? What happens if the required entry marker is replaced with another value? Does keeping the correct marker authenticate the event's producer?

Level 1: state exactly which bytes are protected

Q04. A lossy value conversion

The baseline sanitizer maps a fraction to its reduced rational string. Compare `Fraction(2, 6)` and the literal string `"1/3"`. Can the stored value distinguish them? What must be retained to reconstruct the original type?

Q05. Colliding mapping keys

Normalize the insertion-ordered mapping `{1: "numeric", "1": "text"}` using the baseline's key stringification. What remains? Explain why no SHA-256 collision is needed for this loss of information.

Q06. Sorted JSON and JCS

Python code-point ordering places U+FB33 before U+1F600. JCS uses UTF-16 code units. Which key comes first under JCS? Give a second difference between the baseline encoder and JCS. Explain why the current branch's float-tag change cannot silently be counted as a historical-baseline result.

Level 2: compare verified snapshots

Q07. Four modifications

Retain an independent anchor `(2, h1)`, where `h1` is the digest of entry index 1. Classify each modification under (a) local reload, (b) the retained count/head comparison, and (c) an independently retained raw-file digest:

- Change entry 0's protected reason but keep its old digest.
- Remove entry 1 while keeping the original header.
- Remove entry 1 and rewrite count/head to `(1, h0)`.
- Change only the header schema and genesis timestamp.

State the hash and custody assumptions behind the classification.

Q08. A legitimate extension

A fully verified three-entry file has head `h2`, while the retained anchor is `(2, h1)`. Does inequality of heads prove tampering? What additional prefix comparison can establish extension of the retained normalized history? What does even that comparison leave unproved?

Q09. Cached evidence

Open a verified two-entry sink. An external writer changes its on-disk reason without rehashing. The old object emits the same tail receipt. Why? Give a correct disk-check sequence and explain its custody/concurrency assumption.

Level 3: failed writes and resource obligations

Q10. An unacknowledged append

An append advances memory from two to three entries; replacement then raises `OSError`. Disk still has two entries. Classify the exception, the old object's three-entry receipt, and the reopened file. What must happen before retrying?

Q11. Frozen outside, mutable inside

The sink returns a frozen dataclass containing a dictionary. A caller edits that dictionary; a later append writes the changed data with the old digest. What can a subsequent reload do? Does a tuple of frozen entries provide transitive immutability?

Q12. Cost of individually rewriting the history

Ignore headers and suppose each record contributes exactly 1,000 bytes. Compute accumulated entry bytes written over 100 individual appends and the final snapshot's entry bytes. Identify two additional operational limits that the checksum chain does not impose. Do not call this calculation a benchmark.

Level 4: a catalog is data, not an admitted repertoire

Q13. Census without promotion

The pinned catalog has 61 entries: 56 labelled `candidate`, 5 labelled `active`, and 49 with placeholder fixture pairs. Compute the non-placeholder count and the exact fractions labelled active and carrying placeholders. How many reproduced or enforced detectors do these counts establish?

Q14. An evidence-named hash

An `evidence_sha256` is SHA-256 of `defect_label:run_id:task_id`. Does it bind a reproduction receipt? Distinguish identity of these encoded identifiers, identity of a raw event, and semantic identity of a defect. Explain what 1,000 copies of one record add to independent evidence.

Q15. One display, two contracts

In one declared relation, self-loops are forbidden; in a separate signed-move grammar, loops are legal. The display `a>a` is retained from both routes. Give the reproduced defect and valid near miss, and explain what prevents a phrase detector from transferring the first route's judgment to the second.

Level 5: decisions, denominators, and actual host action

Q16. Three clonal outcomes

Classify `(True, KillConditionReceipt)`, `(False, None)`, and `(False, KillConditionRefusal(code="INVALID_MATCH_PATTERN"))`. Which result rejects the malformed clone without purging its parent? Does returning a kill record mutate either frozen object or apply a host policy?

Q17. The denominator changes the answer

The explicit trigger interface receives one `(True, False)`, two `(True, True)`, and three `(False, False)` records. Compute its toxicity and compare with inclusive tau = 1/5. Also compute the fraction conditional on a pattern match. Why is neither value automatically the population false-suppression rate on independently valid eligible proposals?

Q18. Corpus mode and no evidence

Exactly one of 100 benign prompts matches in corpus mode. What are `signal1_only_count`, `total_triggers`, and toxicity? Contrast this with an empty explicit trigger list. Can either number certify deployment safety? Does a synthesized `DIAGNOSTIC_ONLY` record by itself change enforcement?

Level 6: exact maintenance arithmetic

Q19. Decay from an already idle rule

A rule has score 4/5, idle count 4, and expiry `half_life_runs=10`. Apply eight further idle runs with tau = 5. Compute completed new intervals, new score, new idle count, and `is_active`. Distinguish decay spacing from expiry.

Q20. Prove batching invariance with its assumptions

Starting at idle zero with score 1 and fixed tau = 5, compare one call for 12 runs with calls for 3, 4, and 5 runs. Compute every interval increment and the final score, then prove the telescoping formula for arbitrary batches. State conditions under which this proof no longer applies.

Q21. Stimulation resets the schedule

An inactive rule has score 1/4 and idle count 12. Apply stimulation with boost 1/5. Compute score, idle count, and activity according to the baseline method. Does this result validate a new detector's admission or empirical usefulness?

Level 7: make a publication claim reviewable

Q22. A minimum reusable-evidence packet

Design a packet for promoting one retained event into a candidate detector. Specify event bytes, source/checker bindings, independent anchor custody, reproduction, a valid near miss, predicate, action, and scope. Mark which items are missing from an identifier-hash-only catalog entry. What receipt would be needed to show a host actually applied an admitted action?

Q23. A held-out experiment

A benign panel contains 1,000 rows generated from 40 base contents. Design a split and denominator that do not present repetitions or training examples as independent safety evidence. State one usefulness outcome and one harmful suppression outcome to measure. What would justify extending a claim to a new domain?

Q24. Release audit

A report says three educational tests passed and eleven production-dependent tests skipped. A PDF compiled and a manifest matched. Write an honest release statement, list the missing execution evidence, and separate byte integrity, source-bound behavior, detector effectiveness, and external Zenodo publication.

Grading and advancement

Each question earns 0-4 points: correct result (1), explicit calculation or counterexample (1), appropriate contract/source assumptions (1), and correct evidence limit (1). Total: 96 points. A level passes when each of its three questions earns at least 3 points. Advancement requires all lower levels to pass. Any claim that a hash authenticates an event, a cached receipt proves disk state, or a decision record proves enforcement must be corrected before advancement. The instructor key supplies acceptable answers and corrections.

Scores assess these exercises only; they confer no operational authority, historical Landau credential, or measured learning gain.